

Graph of Convex Sets Trajectory Optimization for Ground-Aerial Bimodal Robots in Point Cloud Environments

Deon Petrizzo

11 June 2026

Abstract

We study the problem of generating smooth, collision-free trajectories for a robot navigating a cluttered environment described only by a raw point cloud, posing it as a convex optimization. The difficulty is twofold: extracting a guaranteed-safe convex representation of free space from unstructured sensor data, and optimizing a continuous trajectory through that space that respects real kinematic limits. We grow obstacle-free convex polytopes directly on the cloud via sphere flipping, assemble them into a Graph of Convex Sets (GCS) along a nominal graph-search path, and optimize a composite Bézier trajectory—a geometric curve composed with a timing curve—as a single convex program. For the special class of robots that exhibit differentially flat dynamics, the output trajectory determines its full motion. Unlike the short-horizon model-predictive control commonly used for local trajectory generation on dynamically complex platforms, this formalizes longer-horizon planning over the robot’s full sensor range as one convex program that is collision-free by construction and respects a velocity limit in physical units, which is particularly attractive for online planning on multimodal ground robots (e.g., M4).

1 Introduction

Mobile robots operating in unstructured environments must continually convert noisy range measurements into motions that are both safe and kinematically reasonable. A widely used paradigm is the *hard-constrained* approach: carve the free space into convex regions (a “safe corridor”), then optimize a trajectory confined to that corridor, so that collision avoidance reduces to convex membership constraints rather than nonconvex distance constraints. This decomposition makes the trajectory problem tractable, but its quality hinges on two sub-problems that are each nontrivial. First, the convex regions must be extracted efficiently and with a guarantee of being obstacle-free, ideally straight from the point cloud so that no expensive intermediate map is maintained. Second, the choice of which regions to route through is a discrete decision coupled

to the continuous trajectory shape, a combination that classical methods handle only heuristically.

These two sub-problems are the focus of this work, scoped to multi-modal robots with a differential-drive mode. The scoping is deliberate. For platforms with complex hybrid dynamics, such as legged robots, local trajectory generation is typically delegated to model-predictive control (MPC) over short horizons, re-solved every control cycle [1]. A differential-drive robot, by contrast, is differentially flat [2]: a smooth trajectory of its planar position determines the full body pose, the forward speed, and the wheel commands. Trajectory generation can therefore be posed directly as a single convex optimization over a *longer* horizon, using the robot’s full sensor range rather than a receding window. This is attractive for online planning, and especially for emerging multimodal platforms that can switch between wheeled driving and aerial flight—for example Caltech’s M4 morphobot [3]—where a longer-horizon planning layer that does not depend on per-cycle MPC is desirable. We build such a kinematic optimizer and structure it so that terrain awareness and discrete mode selection can be layered on (Sec. 4).

2 Related Work

Free-space generation. IRIS [4] inflates a large convex region around a seed using alternating semidefinite and quadratic programs; it produces high-quality polytopes but is iterative and relatively slow. Liu et al. [5] build a safe flight corridor by inflating ellipsoids along a prior path, which is faster but requires a leading path and yields one region per path segment. We instead adopt the sphere-flipping method of Zhong et al. [6], which maps the points through a nonlinear inversion (originating in the hidden-point-removal operator of Katz et al. [7]), takes a convex hull [8], and trims the result to a guaranteed-free polytope. Its advantage is speed (milliseconds for thousands of points) and that it operates directly on the raw cloud with a single parameter; its disadvantage is that the convexification can sacrifice volume relative to IRIS, and it offers no volume-maximality guarantee.

Trajectory optimization. We formulate routing and trajectory shaping with Graphs of Convex Sets [9, 10], whose motion-planning specialization [11] jointly optimizes the region sequence and the continuous trajectory through a tight convex relaxation. Its advantage over corridor methods that fix the sequence a priori is globally informed routing; its disadvantage is that solving the full mixed-integer program is comparatively heavy. Following the convex-restriction idea of the GCS motion chapter [10], we keep the convex Bézier trajectory machinery but resolve the discrete sequence with a lightweight graph search, trading global optimality of the route for speed. A complementary line of work generates local trajectories online with model-predictive control, e.g. convex MPC for quadrupedal locomotion [1], which re-solves a short-horizon problem each control cycle; this is the standard tool when dynamics are complex enough that only a receding window is tractable.

3 Methods

3.1 Problem statement (GCS)

We use the Graph-of-Convex-Sets formulation [9, 10]. Let $G = (\mathcal{V}, \mathcal{E})$ be a directed graph in which every vertex $v \in \mathcal{V}$ carries a compact convex *safe set* $\mathcal{X}_v \subseteq \mathbb{R}^d$ and a continuous variable $x_v \in \mathcal{X}_v$, and each edge $e = (u, v) \in \mathcal{E}$ carries a convex, nonnegative length $\ell_e(x_u, x_v)$. Given a source s and target t , the shortest-path problem in GCS is

$$\begin{aligned} \min_{x, y} \quad & \sum_{e=(u,v) \in \mathcal{E}} \ell_e(x_u, x_v) y_e \\ \text{s.t.} \quad & y \in \{0, 1\}^{\mathcal{E}} \text{ encodes an } s\text{-}t \text{ path (flow conservation),} \\ & x_v \in \mathcal{X}_v, \quad \forall v \in \mathcal{V}, \end{aligned} \tag{1}$$

where the binaries y_e select the edges of the path. Relaxing $y_e \in [0, 1]$ with perspective edge costs gives a tight convex relaxation that can be rounded to a feasible path [11]. For motion planning ([10], Ch. 11), the safe sets $\mathcal{X}_v$ are obstacle-free convex regions of the robot’s output space, the point variable in each region is replaced by a Bézier curve segment, and ℓ_e encodes trajectory length, energy, or duration together with continuity across adjacent regions. We solve a *convex restriction* of (1): the region sequence is fixed by a graph search and the remaining trajectory problem is convex.

For a differentially-driven robot the relevant output space is the planar position. A smooth output trajectory $r(t)$ recovers the body pose, the forward speed $v = \|\dot{r}\|$. Consequently the velocity constraint in our trajectory optimization (Sec. 3.4) bounds the body speed directly. Acceleration and jerk constraints are in general nonconvex in this formulation so are handled as additional weighted penalties in the objective function.

3.2 Pipeline

Information flows from the point cloud to the optimized trajectory in four stages (Fig. 1):

1. **Region growing.** A nominal collision-free path $\{p_q^{(1)}, \dots, p_q^{(m)}\}$ is assumed available from a graph search (A*/PRM) over free-space samples. At each p_q we grow one obstacle-free convex polytope from the local cloud (Sec. 3.3).
2. **Graph assembly.** The polytopes become the vertices $\mathcal{V}$ of a GCS; an edge is added between two regions iff they intersect (a small linear-program feasibility test), giving the safe sets $\mathcal{X}_v$ and edges $\mathcal{E}$.
3. **Routing.** The region sequence is found by Dijkstra’s algorithm on the intersection graph with start/goal wired in as virtual nodes and edge weights given by inter-region Chebyshev-center distances. This rounding step fixes the discrete part of (1).

4. **Trajectory optimization.** With the sequence fixed, one geometric and one timing Bézier curve are assigned per region and the composite trajectory is optimized as a single convex program (Sec. 3.4).

3.3 Convex region generation from point clouds

For a query coordinate p_q and a local cloud $P = \{p_i\}$ we follow the sphere-flipping construction [6]. With the origin shifted to p_q , each point is mapped by

$$p'_i = f(p_i) = p_i \frac{2R - \|p_i\|}{\|p_i\|} \quad \Rightarrow \quad \|p'_i\| = 2R - \|p_i\| \quad (2)$$

so points near p_q are thrown far out. R is the only parameter, encoding the radius of the spherical region we radially reflect points over. Because $\|p'_i\| = 2R - \|p_i\|$, the map is an *involution* and its inverse has the same form as (2). The convex hull of the flipped points [8], mapped back via the inverse sphere flip, bounds a *star-convex* obstacle-free polytope. We trim it to a strictly convex H -polytope $Ax \leq b$: for each hull facet with outward normal n_i , forming a simplex with p_q , we push the supporting hyperplane to the furthest contained star vertex, $b_i = \max_{v \in \text{simplex}} n_i^\top v$. Finally we add a safety pass that tightens each offset against the nearest local cloud point assigned to that facet, which guarantees the region excludes every sampled obstacle point while keeping p_q interior. We chose this method over IRIS [4] and ellipsoid inflation [5] because it is the only one that runs directly on the raw cloud on the order of milliseconds and attaches region vertices to obstacle samples, which is convenient for later deformation under SLAM updates if this algorithm is run online.

3.4 Composite Bézier trajectory optimization

To make velocity and acceleration physically meaningful we use the joint shape-and-timing parameterization of [10] (Sec. 11.4.1). Over a fixed sequence of regions $C_0, \dots, C_{L-1}$, with $C_k = \{x : A_k x \leq b_k\}$, each region k is given a geometric curve q_k and a timing curve h_k , both degree- N Bézier in the path parameter $s \in [0, 1]$,

$$q_k(s) = \sum_{n=0}^N \beta_n^N(s) \mathbf{q}_{k,n}, \quad h_k(s) = \sum_{n=0}^N \beta_n^N(s) h_{k,n}, \quad \beta_n^N(s) = \binom{N}{n} s^n (1-s)^{N-n},$$

with $\mathbf{q}_{k,n} \in \mathbb{R}^d$, $h_{k,n} \in \mathbb{R}$. The physical trajectory is the composition $r(t) = q_k(h_k^{-1}(t))$; by the chain rule $\dot{r} = q'_k/h'_k$, so a real-units speed cap $\|\dot{r}\| \leq v_{\max}$ becomes the second-order cone $\|q'_k(s)\| \leq v_{\max} h'_k(s)$. Derivative control points follow the finite-difference rule $g_n^{(1)} = N(g_{n+1} - g_n)$, with endpoints $g^{(r)}(0) = g_0^{(r)}$ and $g^{(r)}(1) = g_{N-r}^{(r)}$, and the squared- $\mathcal{L}_2$ norm of any derivative is a quadratic form $\sum_{n,m} M_{nm}^{(p)} g_n \cdot g_m$ with the PSD energy matrix $M_{nm}^{(p)} = \binom{p}{n} \binom{p}{m} / [(2p+1) \binom{2p}{n+m}]$. Writing $T := h_{L-1,N}$ for the total duration,

the optimizer solves

$$\begin{aligned}
& \min_{\{\mathbf{q}_{k,n}\}, \{h_{k,n}\}} w_t T + \sum_{k=0}^{L-1} \sum_{n,m} \left(w_q M_{nm}^{(N-2)} \mathbf{q}_{k,n}^{(2)} \cdot \mathbf{q}_{k,m}^{(2)} + w_h M_{nm}^{(N-2)} h_{k,n}^{(2)} h_{k,m}^{(2)} \right) \\
& \text{s.t. } A_k \mathbf{q}_{k,n} \leq b_k \quad (\text{region containment}) \\
& \quad h_{k,n}^{(1)} \geq \underline{h} > 0 \quad (\text{monotone, invertible time}) \\
& \quad \left\| \mathbf{q}_{k,n}^{(1)} \right\| \leq v_{\max} h_{k,n}^{(1)} \quad (\text{velocity limit, SOC}) \\
& \quad \mathbf{q}_{k,N-r}^{(r)} = \mathbf{q}_{k+1,0}^{(r)}, \quad h_{k,N-r}^{(r)} = h_{k+1,0}^{(r)} \quad (\text{continuity, } r = 0, \dots, \kappa) \\
& \quad \mathbf{q}_{0,0} = x_{\text{start}}, \quad \mathbf{q}_{L-1,N} = x_{\text{goal}}, \quad h_{0,0} = 0 \quad (\text{boundary}) \\
& \quad \mathbf{q}_{0,0}^{(1)} = v_0 h_{0,0}^{(1)}, \quad \mathbf{q}_{0,0}^{(2)} = \mathbf{0}, \quad \mathbf{q}_{L-1,N-2}^{(2)} = \mathbf{0} \quad (\text{rest-to-rest}) \\
& \quad (3)
\end{aligned}$$

over the index ranges $n = 0, \dots, N$ (containment), $n = 0, \dots, N-1$ (timing/velocity), and $k = 0, \dots, L-2$ (continuity). Problem (3) is a second-order cone program: the objective is convex quadratic ($M^{(p)} \succeq 0$) and every constraint is linear except the velocity cones. The boundary-velocity equality is linear because v_0 is a constant, and at rest ($v = 0$) zero acceleration reduces to the convex condition $q'' = \mathbf{0}$.

Design choices. We deliberately minimize *time* (dominant weight w_t): the timing curve is what makes a real velocity bound expressible, and a time-optimal traversal under that bound is the quantity an online motion planner cares about, so the objective is what motivates the shape plus timing parameterization. The full composition acceleration $\ddot{r} = (q''h' - q'h'')/h'^3$ is *nonconvex* in $(\mathbf{q}, h)$, so it is not hard-constrained: following [10], we can indirectly shape acceleration and jerk by considering their weighted norms as penalties in the objective. Finally, we solve the convex restriction of (1) rather than the full mixed-integer GCS: fixing the route with a graph search costs us route-optimality but keeps each solve a single fast SOCP, which suits online replanning.

4 Results

We implemented the pipeline in Python using `cvxpy` [12] with the CLARABEL conic solver, `scipy` for the convex hull (Quickhull [8]) and the routing LPs/Dijkstra, and validated it on a synthetic 2D scene (22×14 units, five rectangular obstacles sampled as a ~ 480 -point perimeter cloud) running on a consumer laptop CPU (see Figure 2). This is an initial simple demonstration, but the method is written dimension-generically, so it applies unchanged to a 3D output space (e.g. a ground robot with terrain height, or an aerial mode). From a 9-waypoint nominal path (densified to 25 query points) it grew 25 guaranteed-free convex regions and routed a 14-region sequence from start to goal.

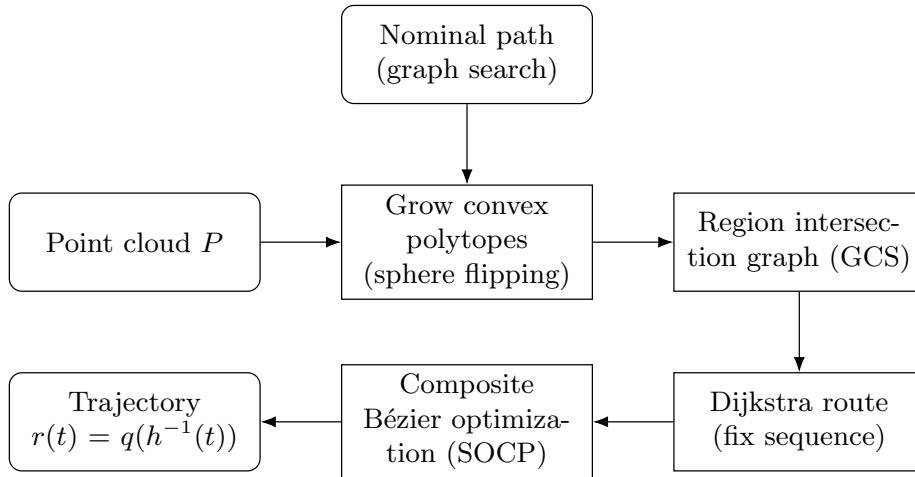


Figure 1: Information flow of the pipeline, from raw point cloud to optimized physical trajectory.

The composite optimizer (3) solved a minimum-time problem with a $v_{\max} = 2.0$ m/s cap on the body speed, rest-to-rest boundary conditions, and C^2 continuity in a few tens of milliseconds. The resulting trajectory was verified collision-free (every sample inside the region union), respected the speed limit exactly (peak ≈ 2.0 m/s), started and stopped at rest, and completed in $T \approx 18.7$ s with a peak acceleration of ≈ 5.7 m/s² concentrated at the accelerate-from-rest phase. The speed profile rides the cap and dips only where path curvature requires, which is the expected time-optimal behavior. Region generation matched the millisecond-scale efficiency reported for sphere flipping [6].

Offline vs. online use. As an *offline* planner the method is broadly useful: for essentially any system it produces high-quality, kinematically reasonable reference trajectories for a lower-level controller to track. Its distinctive value is in *online* planning, where it is most suitable for differentially flat robots that can plan a long horizon over their full sensor range in a single convex solve, rather than relying on a receding-horizon MPC layer as dynamically complex systems must [1].

Terrain-aware safe sets and multimodal mobility. The most promising extension targets uneven and off-road terrain. In the current implementation, the regions are grown purely from obstacle geometry; integrating online local *traversability* analysis [13, 14]—an estimate of drivability from slope, roughness, and surface properties—as an additional indicator in the safe-set generation would let the same machinery plan over terrain, gating which cloud points and regions are admissible or weighting the GCS edges by traversal risk. This is especially compelling for multimodal platforms such as the M4 morphobot [3],

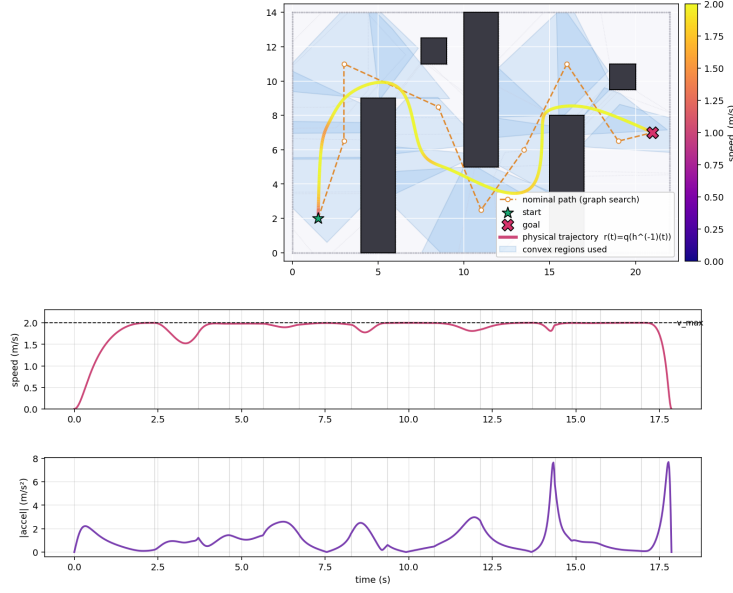


Figure 2: 2D demonstration of trajectory optimizer pipeline.

which can transition between wheeled driving and aerial flight. A natural fit is to attach a discrete *mode* to each GCS vertex (ground-drivable vs. flight); the discrete GCS layer would then choose where to keep driving on traversable ground—typically cheaper and more efficient—before committing to a transition to flight over impassable terrain. Because such a planner exploits the full sensor range in one longer-horizon convex program, it offers a planning layer for these systems that does not depend on MPC.

References

- [1] J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim, “Dynamic locomotion in the MIT Cheetah 3 through convex model-predictive control,” in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 1–9.
- [2] M. Fliess, J. Lévine, P. Martin, and P. Rouchon, “Flatness and defect of non-linear systems: introductory theory and examples,” *International Journal of Control*, vol. 61, no. 6, pp. 1327–1361, 1995.
- [3] E. Sihite, A. Kalantari, R. Nemovi, A. Ramezani, and M. Gharib, “Multi-modal mobility morphobot (M4) with appendage repurposing for locomotion plasticity enhancement,” *Nature Communications*, vol. 14, no. 1, p. 3323, 2023.

- [4] R. Deits and R. Tedrake, “Computing large convex regions of obstacle-free space through semidefinite programming,” in *Algorithmic Foundations of Robotics XI*. Springer, 2015, pp. 109–124.
- [5] S. Liu, M. Watterson, K. Mohta, K. Sun, S. Bhattacharya, C. J. Taylor, and V. Kumar, “Planning dynamically feasible trajectories for quadrotors using safe flight corridors in 3-d complex environments,” *IEEE Robotics and Automation Letters*, vol. 2, no. 3, pp. 1688–1695, 2017.
- [6] X. Zhong, Y. Wu, D. Wang, Q. Wang, C. Xu, and F. Gao, “Generating large convex polytopes directly on point clouds,” *arXiv preprint arXiv:2010.08744*, 2020.
- [7] S. Katz, A. Tal, and R. Basri, “Direct visibility of point sets,” in *ACM SIGGRAPH 2007 Papers*, 2007, pp. 24–es.
- [8] C. B. Barber, D. P. Dobkin, and H. Huhdanpaa, “The quickhull algorithm for convex hulls,” *ACM Transactions on Mathematical Software*, vol. 22, no. 4, pp. 469–483, 1996.
- [9] T. Marcucci, J. Umenberger, P. Parrilo, and R. Tedrake, “Shortest paths in graphs of convex sets,” *SIAM Journal on Optimization*, vol. 34, no. 1, pp. 507–532, 2024.
- [10] T. Marcucci, “Graphs of convex sets with applications to optimal control and motion planning,” Ph.D. dissertation, Massachusetts Institute of Technology, 2024.
- [11] T. Marcucci, M. Petersen, D. von Wrangel, and R. Tedrake, “Motion planning around obstacles with convex optimization,” *Science Robotics*, vol. 8, no. 84, p. eadf7843, 2023.
- [12] S. Diamond and S. Boyd, “CVXPY: A Python-embedded modeling language for convex optimization,” *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.
- [13] P. Papadakis, “Terrain traversability analysis methods for unmanned ground vehicles: A survey,” *Engineering Applications of Artificial Intelligence*, vol. 26, no. 4, pp. 1373–1385, 2013.
- [14] D. D. Fan, K. Otsu, Y. Kubo, A. Dixit, J. Burdick, and A.-A. Agha-Mohammadi, “STEP: Stochastic traversability evaluation and planning for risk-aware off-road navigation,” in *Robotics: Science and Systems (RSS)*, 2021.